exceeding the maximum idle_session_timeout value, a transaction opened with an BEGIN but idle was not terminated.

Per *How to Monitor PostgreSQL Connections* from EnterpriseDB, clients are responsible for closing connections.

> It is worth mentioning that if the connection is not explicitly closed by the application, it will remain available, thereby consuming resources -— even when the client has disconnected.[4]

Setting Active Record Pool Size

The pool size should be set with care to not exceed the maximum number of connections PostgreSQL can handle. The max_connections value should reflect a safe maximum possible value. Active Record will use *up to* the pool size of connections. Connections will be held open and idle for up to 5 minutes by default.

The article *Concurrency and Database Connections in Ruby with ActiveRecord*[5] from Heroku shows how to use an environment variable to control the value for pool to make it easier to change on the fly.

From *Finding and killing long running queries on PostgreSQL*,[6] run the following query to list queries that have been running for more than 5 minutes. View the duration column to see how long they've been running. Run the following query from psql. There will need to be active queries to display any results.

sql/long_running_queries_duration.sql

```
SELECT
  pid,
  NOW() - pg_stat_activity.query_start AS duration,
  query,
  state
FROM pg_stat_activity
WHERE (
    NOW() - pg_stat_activity.query_start
) > INTERVAL '5 minutes';
```

Using the Active Record Connection Pool

Active Record provides an application-level Database Connection Pooling (See: *Database Pooling*[7]) capability.

4. https://www.enterprisedb.com/postgres-tutorials/how-monitor-postgresql-connections
5. https://devcenter.heroku.com/articles/concurrency-and-database-connections
6. https://medium.com/little-programming-joys/finding-and-killing-long-running-queries-on-postgres-7c4f0449e86d
7. https://guides.rubyonrails.org/configuring.html#database-pooling